Health Assistance System

Comprehensive Java Programming Project Documentation

Course Code:	ICT2223
Course Title:	Java Programming 2
Instructor:	Mughe Godlove
Group Number:	32
GitHub Repo:	HealthAssistanceSystem
Date:	05/16/2026

SN	Member's Name	Reg. Number	Role
1	Kamdeu Yamdjeuson Neil Marshall	ICTU20241386	Scrum Master
2	Lemma Precious	ICTU20241658	Product Owner

ABSTRACT

The **Health Assistance System** is an innovative, community-driven software platform developed as part of the ICT2223 Java Programming 2 course. The system addresses the growing need for accessible, digital healthcare assistance by providing users with tools to manage their personal health data, consult symptom checkers, schedule medical appointments, receive medication reminders, and access curated health information resources — all from a unified, user-friendly interface.

Developed following the Scrum agile methodology, the project progresses through five structured phases: team organization, requirements elicitation, architectural and UML design, implementation, and testing with deployment. The system employs a layered MVC architecture that separates the JavaFX screens, controller logic, DAO access, and database models.

This documentation presents the complete lifecycle of the Health Assistance System — from problem definition and use-case modelling to system architecture, UML diagrams, implementation details, and testing results. It serves as the official project report fulfilling all requirements set forth in the ICT2223 project specification.

Keywords: Health Management, Java, Scrum, UML, MVC Architecture, JavaFX Desktop Application, Software Engineering, Community Health

CONTENTS

INTRODUCTION

1.1 Brief Overview of the Project

The **Health Assistance System** is a full-stack Java-based application designed to provide individuals with intelligent, accessible, and personalised health management tools. The platform bridges the gap between patients and health information by delivering a comprehensive set of features through a clean, responsive user interface. The system enables users to log symptoms, track health metrics, book appointments, and receive smart health recommendations — empowering communities to make informed health decisions without immediate access to medical professionals.

Application Type:	JavaFX Desktop Application
Technology Stack:	JavaFX, FXML, CSS, JDBC, MySQL
Architecture:	Layered MVC with DAO-based data access
Methodology:	Agile Scrum (6-week sprints)
Target Deployment:	Local Desktop Application Build

1.2 Problem Statement

Access to basic health assistance remains a significant challenge in many communities. Individuals frequently face:

- Long waiting times at healthcare facilities for minor consultations

- Lack of organised personal health records and medication tracking
- Difficulty in monitoring chronic conditions (blood pressure, glucose, etc.)
- Limited access to reliable, contextualised health information
- Poor medication adherence due to lack of reminder systems

The Health Assistance System directly addresses these pain points by delivering a digital health companion accessible anytime, anywhere.

1.3 Objectives and Purpose

1. Provide a centralised platform for personal health record management
2. Offer a symptom checker powered by a curated medical knowledge base
3. Enable online appointment scheduling and management
4. Deliver medication reminders and adherence tracking
5. Facilitate access to verified health education resources
6. Ensure secure, role-based access for patients and healthcare providers

1.4 Key Features and Innovations

Table 1.1: Key System Features

Feature	Description
Role-based Authentication	Secure login with role-specific dashboards (Patient, Doctor, Admin).
Appointment Management	Full CRUD with conflict checks, expiration tracking, and slot reactivation.
Appointment Reminders	Background alerts with JavaFX popups and audio playback (MP3 or system beep).
Health Records	Persistent patient data, medical records, and prescriptions via DAO and MySQL.
Role-based Dashboards	Customised views showing relevant metrics, appointments, and quick actions.
Background Threading	Daemon reminder task with safe JavaFX UI updates via `Platform.runLater()`.

PROJECT MANAGEMENT (SCRUM)

2.1 Team Roles and Responsibilities

Table 2.1: Team Roles and Responsibilities

Member	Role	Responsibilities
Kamdeu Yamdjeuson	Application Logic / Database Dev	Sprint facilitation, DAO development, database design, reminder task logic
Lemma Precious	UI / Workflow Dev	FXML screens, controller logic, validation, and JavaFX interface implementation

2.2 Overview of the Scrum Process

The project adopted the **Scrum framework**, organising work into six weekly sprints aligned with the six project phases. Each sprint followed the standard Scrum ceremonies:

Sprint Planning
Conducted at the start of each week to select and estimate backlog items

Daily Stand-ups
15-minute synchronisation meetings covering: done / doing / blockers

Sprint Review
Demonstration of completed features to the instructor / stakeholders

Sprint Retrospective
Team reflection on process improvement and lessons learned

2.3 Sprint Planning Summary

Table 2.2: Sprint Plan Overview

Sprint	Week	Goal / Deliverables	Status
1	Week 1	Team setup, GitHub repo, project charter	Done
2	Week 2	Requirements elicitation and backlog creation	Done
3	Week 3	HLD architecture, component diagrams	Done
4	Week 4	UML diagrams, core backend implementation	Done
5	Week 5	Frontend development, API integration, module completion	Done
6	Week 6	Testing, deployment, documentation, presentation	Done

2.4 Product Backlog

Table 2.3: Product Backlog

ID	Epic	User Story	Priority	Pts
US01	Authentication	As a user I can register and log in securely	High	5
US02	Profile	As a user I can manage my personal health profile	High	3

(Continued from previous page)

ID	Epic	User Story	Priority	Pts
US03	Symptom Checker	As a user I can enter symptoms and receive possible conditions	High	8
US04	Appointments	As a user I can view available doctors and book appointments	High	8
US05	Appointments	As a doctor I can view and manage my appointment schedule	High	5
US06	Medications	As a user I can log medications and set reminders	Medium	5
US07	Dashboard	As a user I can view my health metrics on a dashboard	Medium	5
US08	Records	As a user I can upload and view my medical records	Medium	3
US09	Library	As a user I can browse health education articles	Low	3
US10	Emergency	As a user I can access emergency contacts quickly	High	2
US11	Admin	As an admin I can manage users and system settings	Medium	5
US12	Notifications	As a user I receive medication and appointment reminders	Medium	5

APPLICATION DESIGN AND ARCHITECTURE

3.1 Overview of Front-End Design

3.1.1 UI/UX Design Principles

The Health Assistance System interface adheres to the following design principles:

- **Simplicity:** Clean, uncluttered layouts reducing cognitive load for health-anxious users
- **Accessibility:** High-contrast colour scheme, readable fonts, and keyboard navigation
- **Consistency:** Uniform component library across all screens (buttons, cards, inputs)
- **Responsive Design:** Adaptive layouts for desktop, tablet, and mobile viewports
- **User Feedback:** Immediate visual confirmation of actions (toasts, progress indicators)

3.2 System Architectural Design

3.2.1 High-Level Architecture Diagram

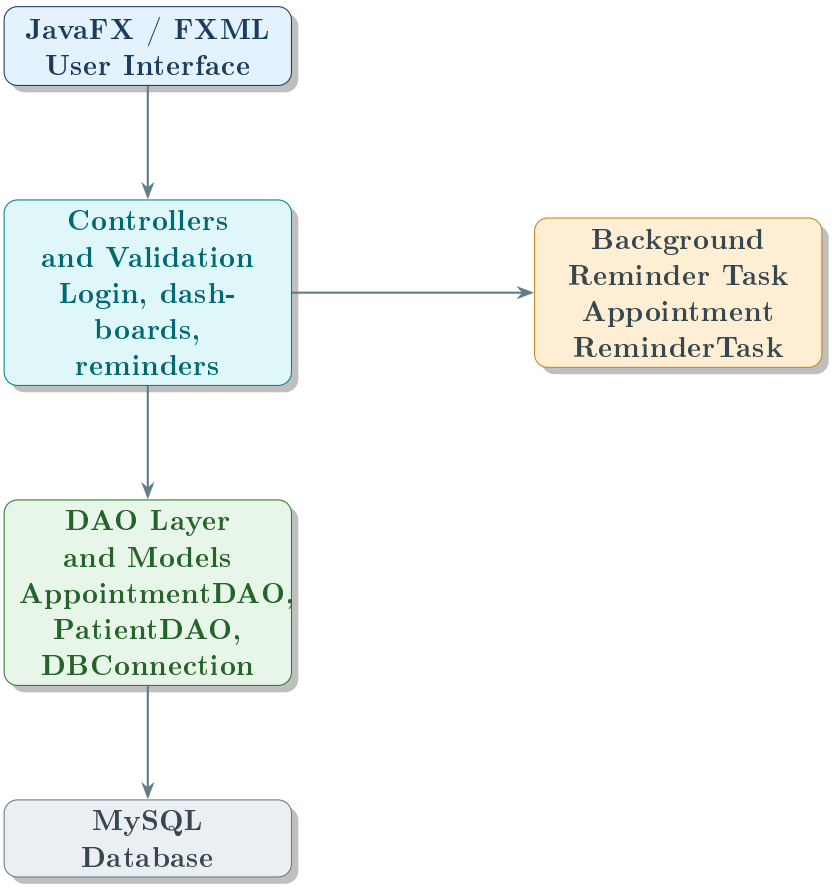

Figure 3.1: High-Level System Architecture

3.2.2 Architecture Description

The Health Assistance System adopts a **Layered MVC Architecture** organised into four practical layers:

Presentation Layer

JavaFX scenes defined with FXML and styled with CSS.

Controller Layer

Java controller classes handle user actions, validation, and navigation.

Data Access Layer

DAO classes and `DBConnection` manage JDBC queries and database operations.

Data Layer

MySQL stores users, appointments, doctors, patients, and health records.

3.2.3 Non-Functional Requirements from Architecture

Table 3.1: Non-Functional Requirements

NFR	How Architecture Addresses It	Target
Security	Role-based login validation and form checks	Good
Scalability	Modular controllers and DAOs make features easy to extend	Good
Performance	Lightweight desktop app with direct JDBC access	Fast response
Availability	Runs locally when the application is open	High during use
Maintainability	MVC separation and reusable DAO classes	High
Data Integrity	SQL constraints and input validation protect records	High

3.3 Low-Level Design – UML Diagrams

3.3.1 Use Case Diagram

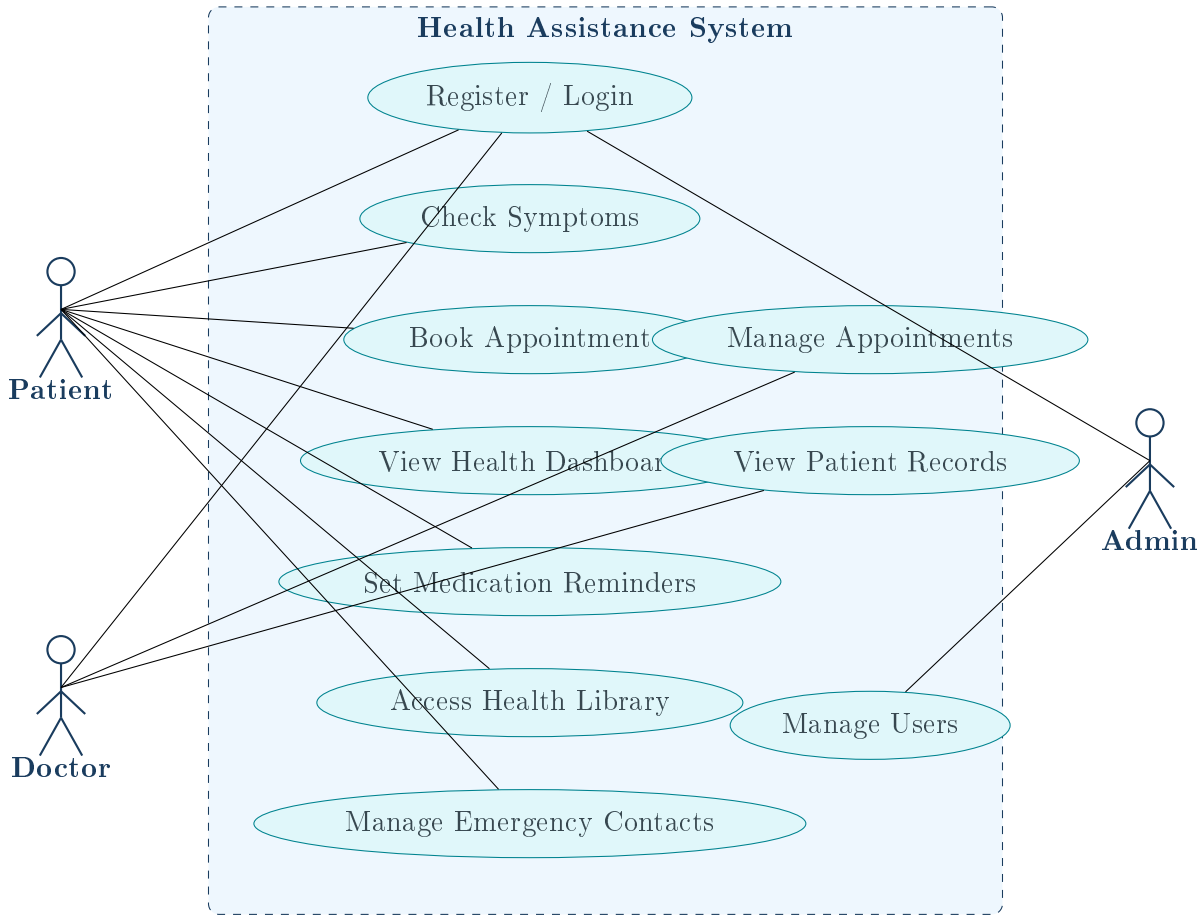

Figure 3.2: System Use Case Diagram

3.3.2 Class Diagram

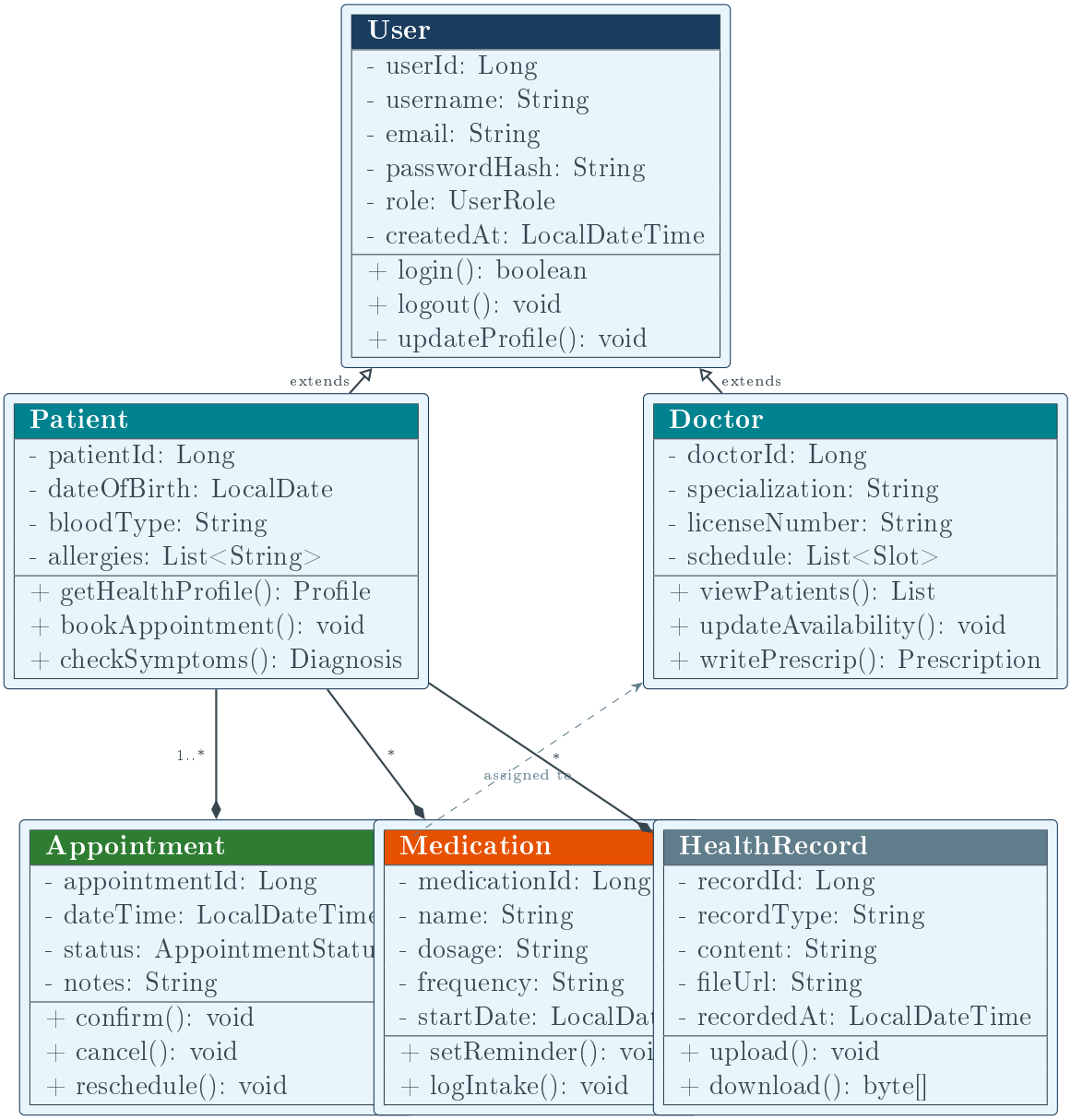

Figure 3.3: System Class Diagram

3.3.3 Sequence Diagrams

SD-1: User Registration and Login

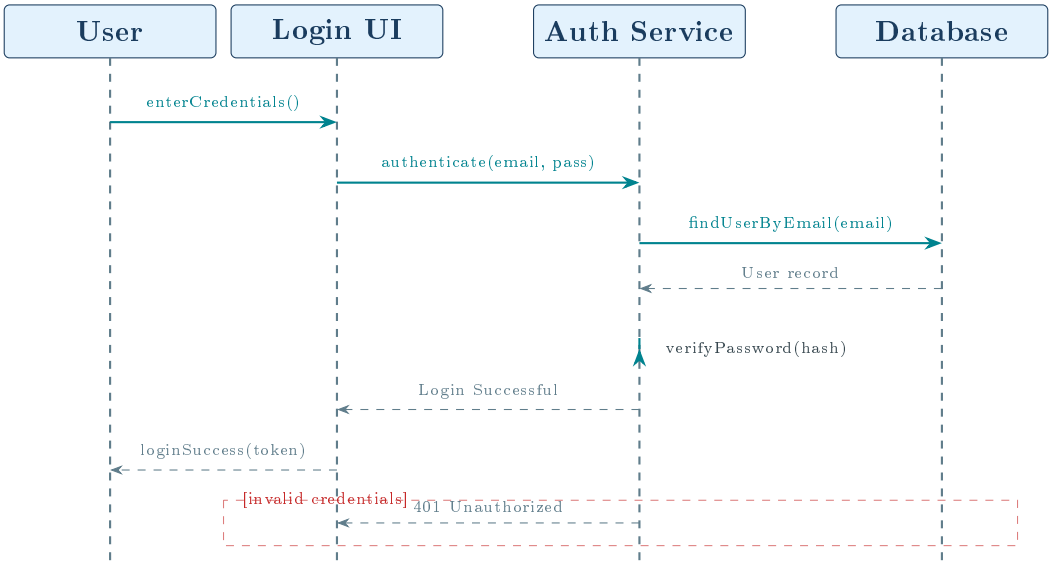

Figure 3.4: SD-1: User Authentication Sequence

3.4 Technology Stack

Table 3.2: Technology Stack

Layer	Technology	Purpose
UI	JavaFX, FXML, CSS	Desktop screens, layout, and styling
Logic	Java Controllers	User actions, validation, and screen flow
Data Access	JDBC / DAO classes	Database queries and persistence
Database	MySQL 8	Persistent relational data storage
Background Tasks	Java threads / Platform.runLater	Appointment reminders and alerts
Build / Run	Java compiler and JavaFX SDK	Compile and launch the desktop application
Version Control	Git + GitHub	Source code management and collaboration
Project Mgmt	Team coordination	Agile sprint and task management

RESULTS AND DISCUSSION

4.1 Application Screenshots

This section presents screenshots of the Health Assistance System across its key functional areas. Replace each placeholder with the actual application screenshots.

4.1.1 Authentication Screens

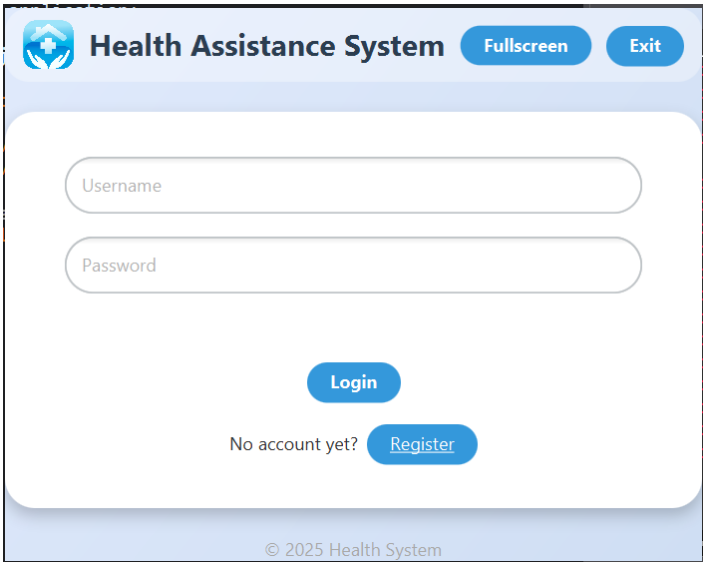

Figure 4.1: Login Screen – User authentication with email/password fields and role-based login

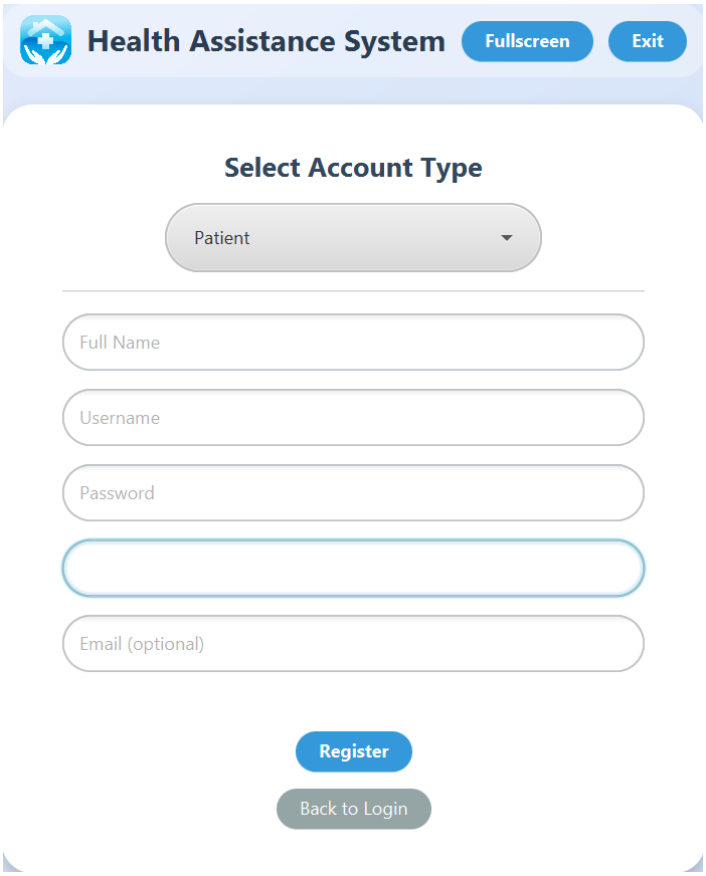

Figure 4.2: Registration Screen – New user registration with field validation and role selection

4.1.2 Patient Dashboard

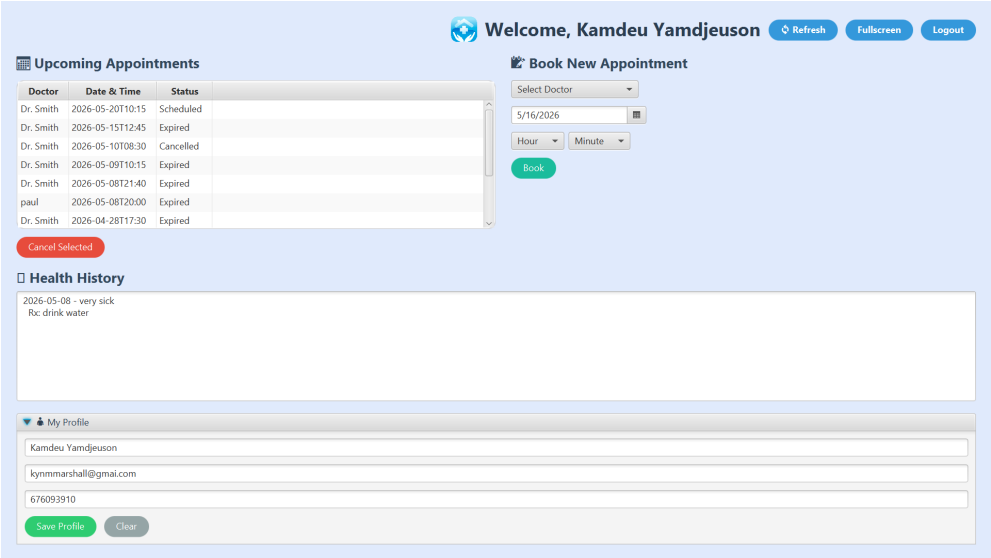

Figure 4.3: Patient Health Dashboard – Overview of health metrics, appointments, medications, and alerts

4.1.3 Doctor Portal

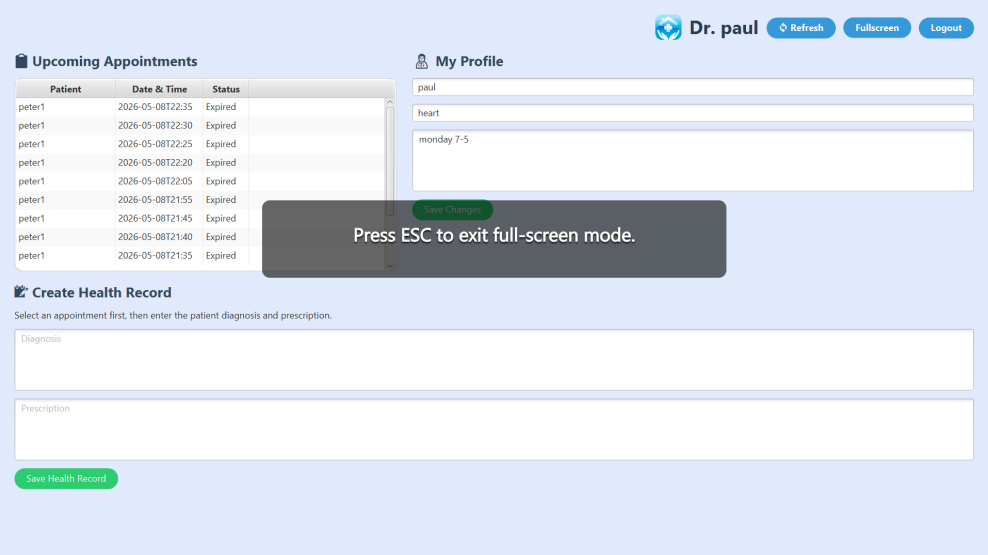

Figure 4.4: Doctor Dashboard – Schedule management, patient records, and clinical overview

4.1.4 Admin Dashboard

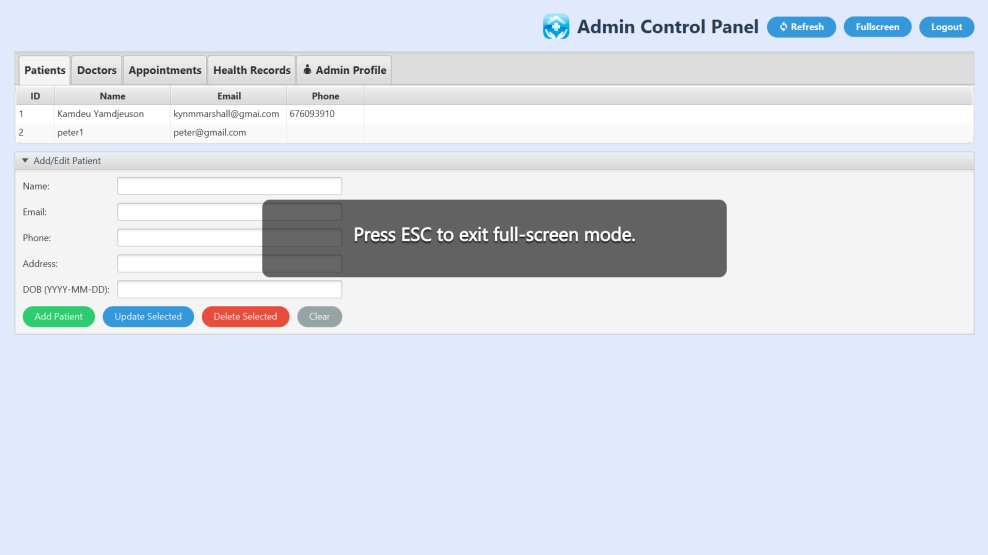

Figure 4.5: Admin Dashboard – System management, user administration, and analytics

CONCLUSION AND FUTURE WORK

5.1 Summary of Project Outcomes

The Health Assistance System successfully delivers a comprehensive, community-focused digital health platform built on solid software engineering principles. All six project phases were completed as scheduled within the Scrum framework, producing a functional application that addresses real-world health management challenges.

- Fully functional JavaFX desktop application with role-based login
- Patient, doctor, and admin dashboards with persistent database-backed data
- Appointment booking system with conflict checking and expiration handling
- Automated appointment reminder with popup alert and looping sound
- Health record and profile management through DAO-based persistence
- Complete UML documentation (use cases, class, 5 sequence, 3 activity, deployment)

5.2 Challenges and Solutions

5.2.1 Technical Challenges

- **Appointment Persistence:** Keeping appointment booking and status changes consistent required using DAO methods with validation and conflict checks.
- **Real-time Notifications:** Scheduling background reminder checks required a daemon thread and safe JavaFX UI updates with `Platform.runLater()`.
- **Database Consistency:** Time-based status updates had to be handled carefully so appointments were not marked expired too early.

5.2.2 Collaboration Challenges

- Coordinating concurrent development on shared modules led to merge conflicts, resolved by adopting a strict feature-branch Git workflow.
- Differing opinions on UI design were resolved through wireframe prototyping and feedback sessions.

5.3 Lessons Learned

1. Clear separation between controller, DAO, and model code makes debugging much easier in a JavaFX project.
2. Scrum ceremonies, though brief, significantly improved team alignment and early identification of blockers.
3. Database time handling is critical; application-side timestamps are more reliable than assuming database time matches local time.
4. UML diagrams produced before coding, not after, genuinely guided implementation decisions.

PROJECT STRUCTURE

The project is organised as a JavaFX desktop application with source code, compiled output, and documentation:

```
HealthAssistanceSystem/
|---- src/
|    |---- application/
|    |---- controller/
|    |---- dao/
|    |---- database/
|    |---- model/
|    |---- resources/
|    |    |---- css/
|    |    '----␣fxml/
|␣␣␣'---- util/
|---- bin/
|---- Documentation/
|---- build.fxbuild
```

Listing 6.1: Project Directory Structure